

Eric Faltermeier 3053092 Monday 8AM 9/20/2026

```

import random
import time
import numpy as np

def main():
    while True:

        # keeps running until the user inputs a 0
        shouldContinue = int(input("Enter 0 to enter an array, or 1 to qu
        if (shouldContinue == 1):
            break

        # gets an input array from the user
        inputArray = list(map(int, input("Enter the unsorted array separa
        print(quickSort(inputArray))

# uses a new array to partition the array and returns it
def newArrayPartition(inputArray):
    arraySize = len(inputArray)
    partitionedArray = []

    if arraySize >= 2:

        # gets the pivot and creates a new empty sorted array
        pivot = inputArray[0]

        # i holds the index for the smaller than pivot portion of the arr
        i = 0

        # itterates through the values of the input array starting after
        for key in inputArray[1:]:

            # if the key is smaller than or equal to the pivot it puts it
            if key <= pivot:
                partitionedArray.insert(i, key)
                i += 1
            # otherwise it adds it to the end
            else:
                partitionedArray.append(key)

        # puts the pivot in its place
        partitionedArray.insert(i, pivot)

        # prints information about the array
        print("Pivot index: ", i)
        print("\nPivot value: ", pivot)
        print("\nLeft part of array" partitionedArray[:i])

```

```

        print("\nRight part of array", partitionedArray[i + 1:])

    # if the array has one element then its already done
    elif arraySize == 1:
        partitionedArray = inputArray

    # returns the sorted array
    return partitionedArray

# partitions the array by shifting items within the same array
def inplacePartition(inputArray):
    # gets the size of the array
    arraySize = len(inputArray)

    if arraySize >= 2:
        pivot = inputArray[0]

        # i contains the index of the left side of the array
        i = 1

        # loops through the array starting after the pivot
        for j in range(1, arraySize):

            # if the key is smaller than the pivot and j and i haven't di
            # swap values until the key is in the right half
            if inputArray[j] < pivot:
                if i != j:
                    # swaps values until the key is in the right spot
                    for k in range(j, i, -1):
                        inputArray[k], inputArray[k - 1] = inputArray[k -

                    # moves the left side of the array
                    i += 1
                # places the key where it belongs
                inputArray[0], inputArray[i - 1] = inputArray[i - 1], inputArray[

    # returns the array and the pivot index
    return inputArray, i - 1

# implements the recursive quicksort algorithm
def quickSort(inputArray):

    # this is the base case where there is one or fewer elements
    if len(inputArray) <= 1:
        return inputArray

    # gets partitioned array and pivot index from inplacePartition
    inputArray, pivotIndex = inplacePartition(inputArray)

    # delineates the halves of the array
    left = inputArray[:pivotIndex]

```

```

right = inputArray[pivotIndex + 1:]

# runs quicksort on the halves
left = quickSort(left)
right = quickSort(right)

# puts back together the halves of the array
return left + [inputArray[pivotIndex]] + right

newArrayPartition([8, 3, 6, 2, 7, 5, 1, 4])
main()

# -----
# Experiment
# -----
# 1. Create a large random array
N = 5000 # you can change size (e.g., 10_000)
arr = [random.randint(0, 10**6) for _ in range(N)]
# 2. Time Quick Sort
arr_copy = arr[:]
start = time.perf_counter()
output = quickSort(arr_copy)
t_quick = time.perf_counter() - start
print(f"Quick Sort time: {t_quick:.6f} sec")
# 3. Verify correctness against numpy sort (and measure numpy time)
arr_copy = arr[:]
start = time.perf_counter()
np_sorted = np.sort(arr_copy) # numpy's highly optimized sort
# (timsort/quick depending on dtype)
t_numpy = time.perf_counter() - start
print(f"NumPy Sort time: {t_numpy:.6f} sec")

```

Pivot index: 7

Pivot value: 8

Left part of array [3, 6, 2, 7, 5, 1, 4]

Right part of array []

Enter 0 to enter an array, or 1 to quit: 0

Enter the unsorted array separated by spaces: 31 15 2 17 19 26 41 17 17
[2, 15, 17, 17, 17, 19, 26, 31, 41]

Enter 0 to enter an array, or 1 to quit: 0

Enter the unsorted array separated by spaces:
[]

Enter 0 to enter an array, or 1 to quit: 0

Enter the unsorted array separated by spaces: 7 7 7 7
[7, 7, 7, 7]

Enter 0 to enter an array, or 1 to quit: 0

Enter the unsorted array separated by spaces: 5 4 3 2 1
[1, 2, 3, 4, 5]

Enter 0 to enter an array, or 1 to quit: 1

```
Quick Sort time: 0.685047 sec  
NumPy Sort time: 0.004388 sec
```